attributes (because `<rect>` elements are drawn from the top left and not from the center like `<circle>` elements) and also apply the width and height values that our layout computes.

Listing 10.9 Appending rectangles with our layout

```
d3.select("g").selectAll("rect")
.transition()
.duration(1000)
.attr("x", function(d) {return d.x - (d.width / 2);})
.attr("y", function(d) {return d.y - (d.height / 2);})
.attr("width", function(d) {return d.width;})
.attr("height", function(d) {return d.height;})
.each("end", resizeGrid1);
```

If we update the rest of our code accordingly, the result is the same animated transition of our layout between different sizes, but now with rectangles that grow and distort based on those sizes, as shown in figure 10.5.

This is a simple example of a layout, and doesn't do nearly as much as the kinds of layouts we've used throughout this book, but even a simple layout like this provides reusable, animatable content. Now we'll look at another reusable pattern in D3, the component, which creates graphical elements automatically.

10.2 Writing your own components

You've seen components in action, particularly the `d3.svg.axis` component. You can also think of the brush as a component, because it creates graphical elements. But it tends to be described as a "control" because it also loads with built-in interactivity.

The component that we'll build is a simple legend. Legends are a necessity when working with data visualization, and they all share some things in common. First, we'll need a more interesting dataset to consider, though we'll continue to use our grid layout. The legend component that we'll create will consist eventually of labeled rectangles, each with a color corresponding to the color assigned to our datapoints by a D3 scale. This way our users can tell, at a glance, which colors correspond to which values in our data visualization.

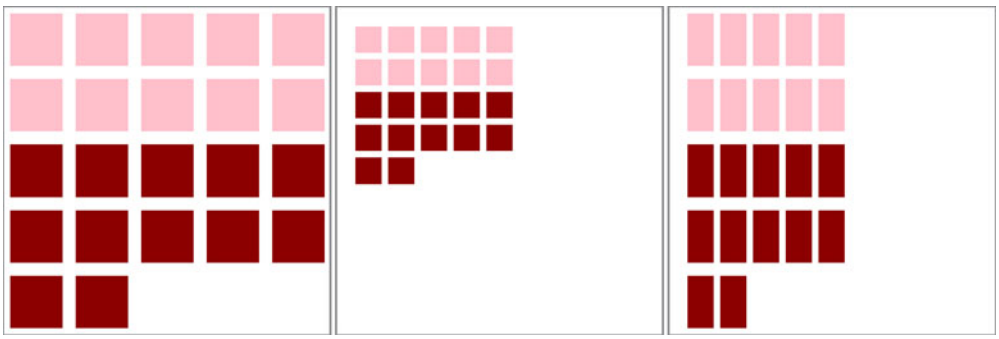


Figure 10.5 The three states of the grid layout using rectangles for the grid cells